

TechBiz Security

Cyber Security Internship Report

Week 5 – Task 13 (EH)

SQL Injection

SQL Injection – Hands-on Testing with DVWA

Submitted By: MANOJ

Role: Cyber Security Intern

Internship Organisation: TechBiz Security

Internship ID: TBS-INT-2026-518

Submitted Date: 17 SEPT 2026

1. Introduction

This report documents the practical work carried out for Week 5 – Task 13 (EH), covering SQL Injection, a web application vulnerability in which improperly handled user input is able to influence the structure or logic of an SQL query executed by the backend database. When user input is concatenated directly into a query without proper validation or parameterization, an attacker can alter the intended query logic, potentially exposing or manipulating data beyond what the application was designed to return.

All practical testing described in this report was performed in an authorized, intentionally vulnerable local lab environment using the Damn Vulnerable Web Application (DVWA). No testing was performed against any live, production, or externally hosted system.

2. Objectives

- Understand the fundamentals of SQL Injection.
- Study SQL Injection through TryHackMe practical learning resources.
- Configure and test SQL Injection in DVWA.
- Test a basic SQL Injection payload.
- Observe whether user-controlled input can alter the application's SQL query behavior.
- Understand the concept of UNION-based SQL Injection.
- Understand database/table/column enumeration concepts.
- Understand the security impact of SQL Injection.
- Document findings and recommend appropriate remediation techniques.

3. Lab Environment

Environment	Kali Linux
Application	Damn Vulnerable Web Application (DVWA)
Target	Local DVWA instance
URL observed in screenshots	http://127.0.0.1:8080/DVWA/
Vulnerability Module	SQL Injection
Testing Type	Authorized local laboratory testing
Security Level	Not visible/confirmed in the provided screenshots

4. Learning Resources and TryHackMe Evidence

The following TryHackMe screenshots were collected in support of the theoretical and guided-practical learning component of this task.

4.1 Locked Room

The screenshot below shows a TryHackMe page stating that the room is locked, with the message “The room owner has locked this room, which means you cannot view any task content.” This screenshot demonstrates a room-access limitation only and is not evidence of completion of any learning content.

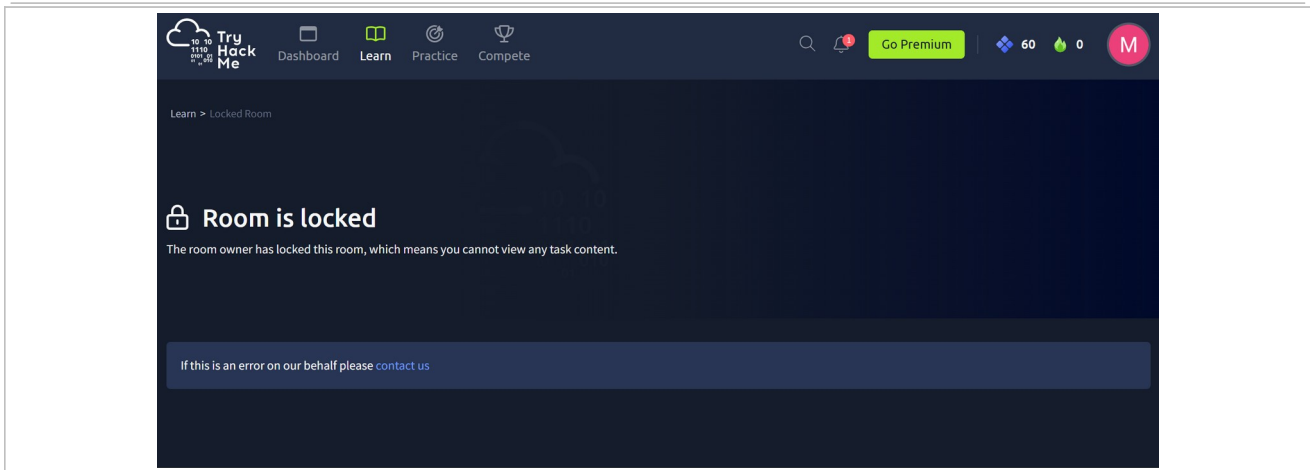


Figure 1: TryHackMe room access status showing the room locked.

4.2 SQL Injection Room – Level 4 Progress

The screenshot below shows the TryHackMe SQL Injection room at 76% progress, with a correct answer submitted for the Level 4 question (“What is the final flag after completing level four?”) and the visible flag THM{SQL_INJECTION_MASTER}.

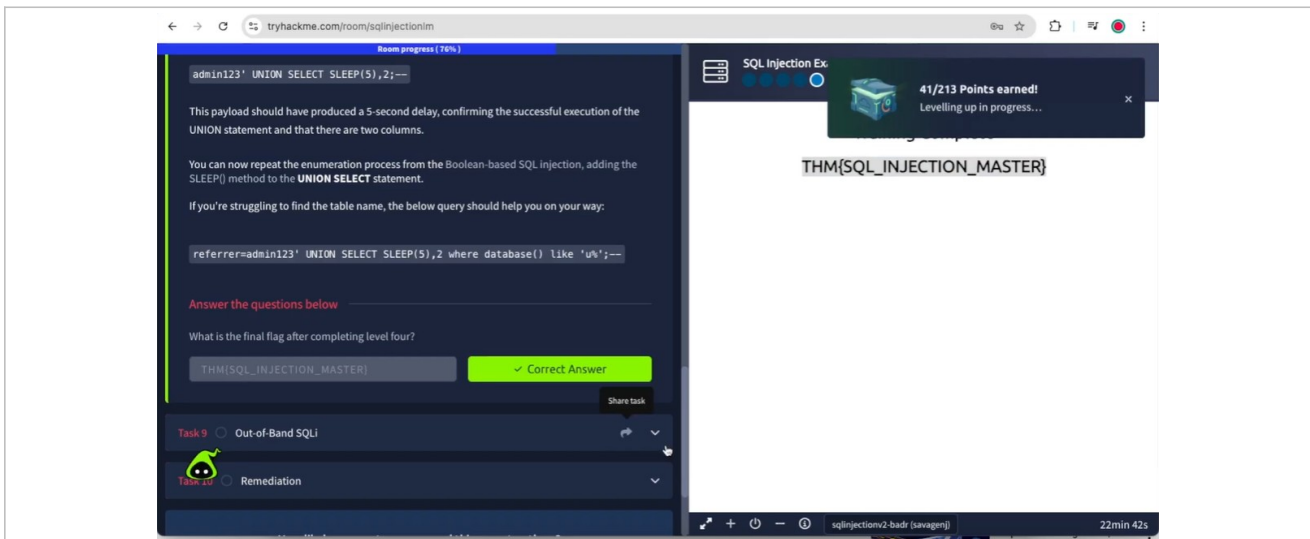


Figure 2: TryHackMe SQL Injection room showing the correct Level 4 answer and visible flag.

4.3 SQL Injection Room – 100% Completion

The screenshot below shows the TryHackMe SQL Injection room with a progress bar indicating “Room completed: 100%”, confirming full completion of the room.

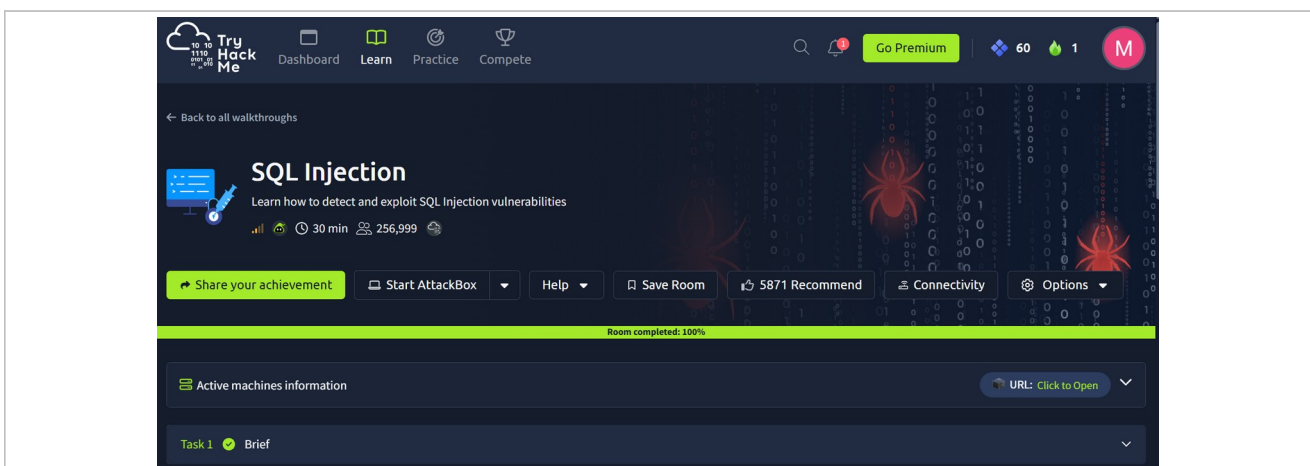


Figure 3: TryHackMe SQL Injection room showing 100% completion.

4.4 OWASP Top 10 2025 – 100% Completion

The screenshot below shows the TryHackMe room “OWASP Top 10 2025: IAAA Failures” with a progress bar indicating “Room completed: 100%”, confirming completion of this OWASP Top 10 2025 learning activity.

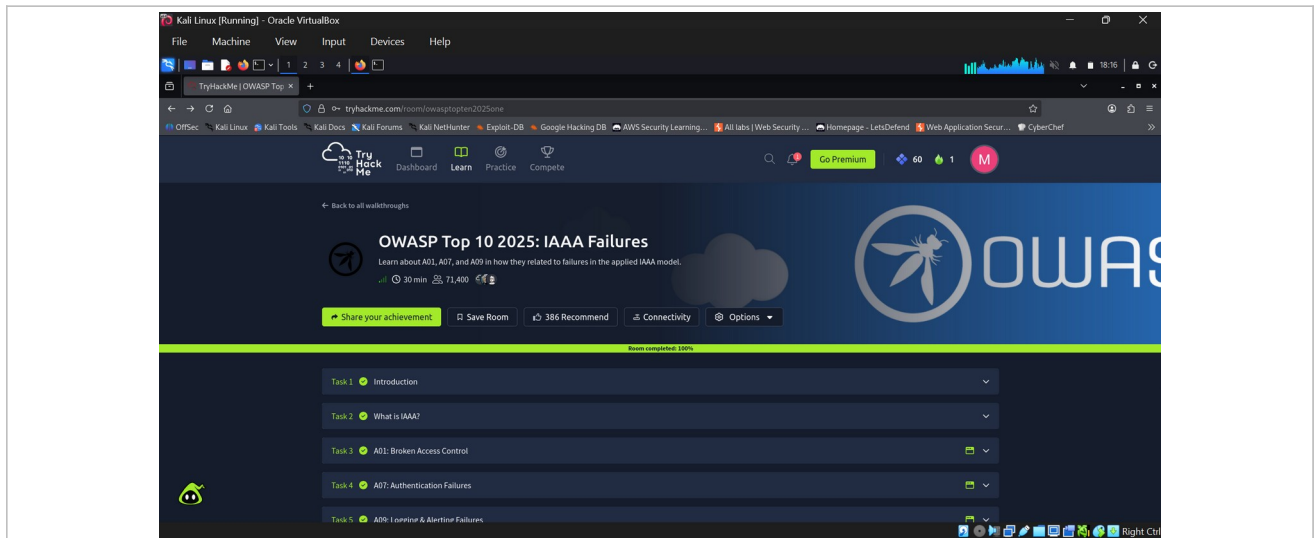


Figure 4: TryHackMe OWASP Top 10 2025 room showing 100% completion.

5. SQL Injection Fundamentals

SQL Injection is a web application vulnerability that occurs when user-supplied input is incorporated into an SQL query without adequate validation or sanitization. Instead of being treated purely as data, the input is interpreted as part of the query's logic by the database engine.

When an application builds a query by directly concatenating user input into the SQL statement, an attacker can craft input that changes the structure of the query – for example, by closing an existing string literal and adding additional logical conditions. If the application does not validate or parameterize this input, the database will execute the modified query as written.

Depending on the application and database configuration, the potential impact of SQL Injection can include unauthorized disclosure of data, bypassing of authentication mechanisms, unauthorized modification of database records, and broader loss of confidentiality and integrity of the affected system. The DVWA SQL Injection module used in this exercise is designed to demonstrate these risks in a safe, intentionally vulnerable environment.

6. DVWA SQL Injection Practical Setup

The practical exercise began by opening the local DVWA application at <http://127.0.0.1:8080/DVWA/> and navigating to the SQL Injection vulnerability module. The module presents a “User ID” input field and a Submit button, which passes the entered value to the backend query.

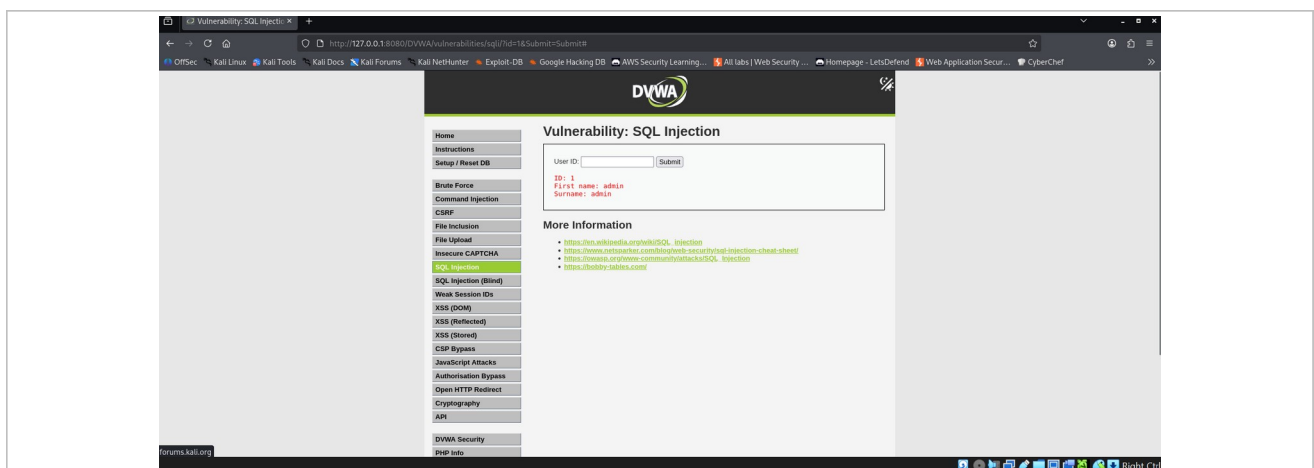


Figure 6: DVWA SQL Injection module showing the observed response.

The screenshot above shows the SQL Injection module in its baseline state, with a submitted User ID of 1 returning a single record (ID: 1, First name: admin, Surname: admin). This establishes the module's normal expected behavior prior to injection testing.

7. Basic SQL Injection Testing

The following payload, visible in the screenshots, was entered into the User ID field:

```
' OR '1'='1
```

This payload attempts to alter the logical condition of the underlying query so that the condition evaluates as true for every row in the users table, rather than only for the row matching a specific ID.

The screenshot below shows the payload entered into the User ID field and submitted.

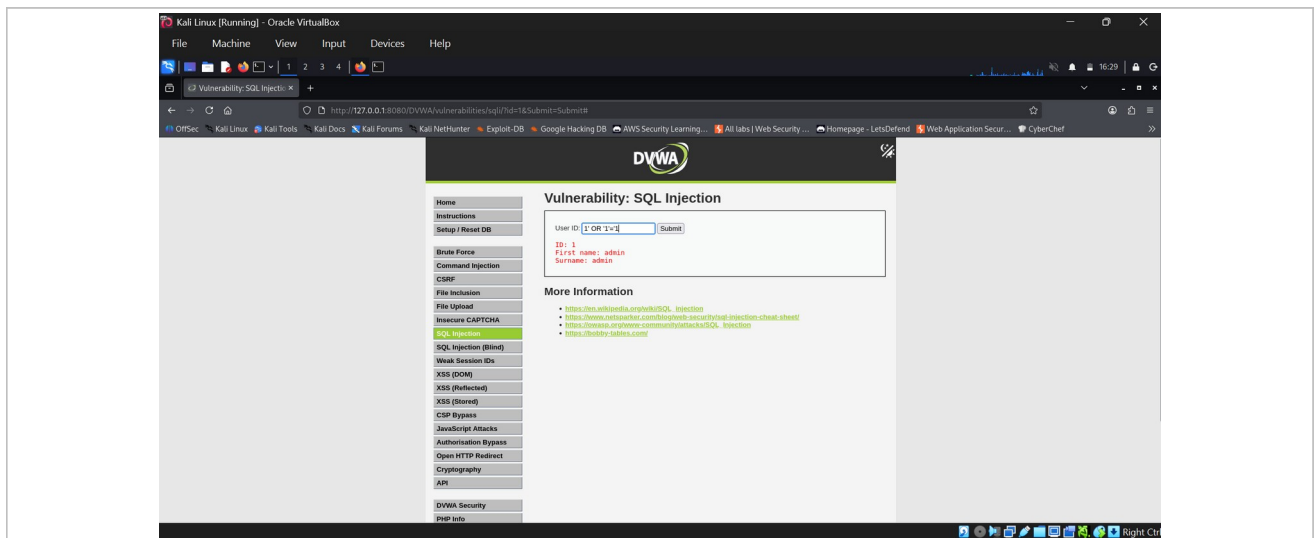


Figure 7: DVWA SQL Injection basic payload submission.

The screenshot below shows the result of the payload against the application: instead of a single record, the application returned multiple user records from the underlying table.

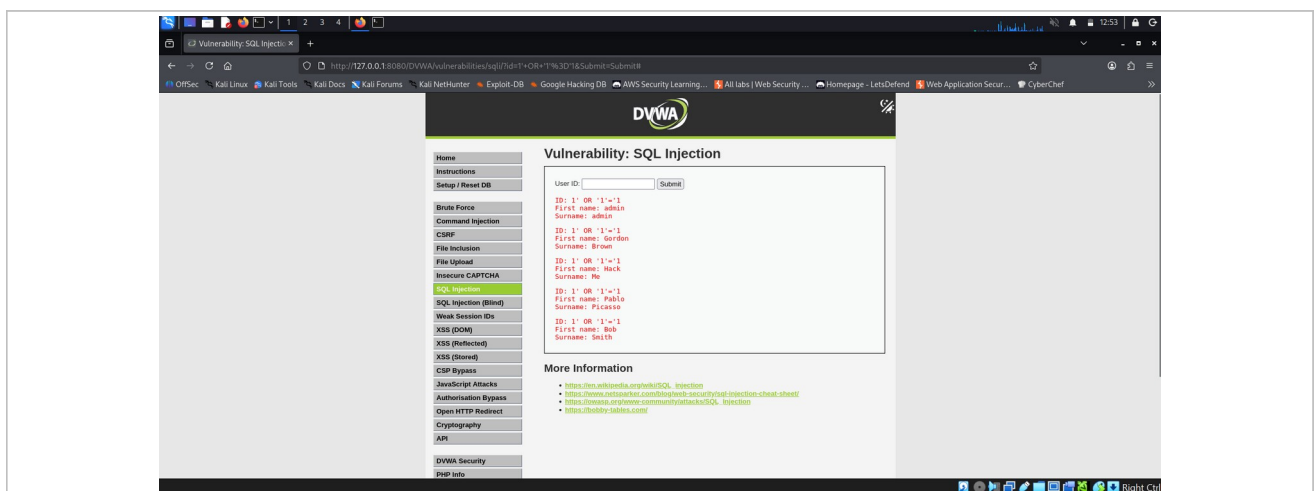


Figure 5: DVWA SQL Injection basic payload resulting in multiple user records.

The visible returned records were:

- ID: 1, First name: admin, Surname: admin
- ID: 2, First name: Gordon, Surname: Brown
- ID: 3, First name: Hack, Surname: Me

- ID: 4, First name: Pablo, Surname: Picasso
- ID: 5, First name: Bob, Surname: Smith

Returning multiple records after submitting the injected input demonstrates that the user-controlled input affected the query's logic in the intentionally vulnerable DVWA application, rather than being treated strictly as a literal ID value.

8. SQL Injection Observation

Test	Payload/Input	Observed Result	Security Significance
Basic SQL Injection	' OR '1'='1	Multiple records returned	Demonstrates that unsanitized input can alter query logic.

9. UNION-Based SQL Injection – Concept

Conceptual learning / methodology: UNION-based SQL Injection is a technique in which an attacker appends a SELECT statement to an existing vulnerable query using the UNION operator, causing the results of the injected query to be combined with the original query's output. Successful use of this technique typically requires first determining the number of columns expected by the original query (for example, using ORDER BY or a UNION SELECT with NULL placeholders), then identifying which columns are reflected in the visible application output, and finally using those columns to enumerate database schema information such as table and column names.

This section is included as conceptual/methodology background only. The provided screenshots do not demonstrate a completed DVWA UNION-based extraction of database table names, column names, usernames, or password hashes, and no such claim is made in this report.

10. Data Extraction Requirement

The task required understanding of how usernames and password hashes could be extracted from the DVWA users table. Based strictly on the evidence provided:

- The supplied screenshots demonstrate successful basic SQL Injection resulting in multi-record disclosure (Figures 5 and 7).
- Specific password-hash extraction is not demonstrated in the supplied screenshots.
- No database, table, or column names were observed being extracted in the supplied evidence, and none are claimed here.

11. Evidence Summary

Screenshot	Evidence Demonstrated
Figure 1 – TryHackMe Locked Room	TryHackMe room access limitation/locked-room message
Figure 2 – TryHackMe SQL Injection Level 4	SQL Injection room progress and correct Level 4 answer with visible flag
Figure 3 – TryHackMe SQL Injection Room Completed	SQL Injection room completed at 100%
Figure 4 – TryHackMe OWASP Top 10 2025	OWASP Top 10 2025 learning room completed at 100%
Figure 5 – DVWA Basic Payload, Multiple Users	Basic SQL Injection payload resulting in multiple returned records
Figure 6 – DVWA SQL Injection	DVWA SQL Injection page showing the baseline observed response

Screenshot	Evidence Demonstrated
Module	
Figure 7 – DVWA Basic Payload Submission	DVWA SQL Injection payload entered and submitted in the User ID field

12. Vulnerability Impact

SQL Injection vulnerabilities can, in general, lead to a range of security impacts depending on the affected application and database configuration:

- Unauthorized database access.
- Disclosure of sensitive information.
- Authentication bypass.
- Modification or deletion of database records.
- Potential compromise of application confidentiality and integrity.
- Possible escalation of impact depending on database privileges and application architecture.

These impacts are described in general terms as characteristics of the SQL Injection vulnerability class. Within this specific lab exercise, only basic query-logic manipulation and multi-record disclosure were demonstrated, as described in Section 7; broader impacts such as authentication bypass or record modification were not attempted or demonstrated.

13. Root Cause

Based on the security principles demonstrated by the vulnerable DVWA application, the likely root causes of SQL Injection in an application of this type include:

- Direct incorporation of user-controlled input into SQL queries.
- Lack of parameterized/prepared statements.
- Insufficient server-side input validation.
- Excessive database privileges, which can increase the impact of a successful injection.

These points reflect general security principles illustrated by DVWA's intentionally vulnerable design, rather than a review of DVWA's actual source code.

14. Remediation

- Use parameterized queries/prepared statements.
- Use safe database APIs.
- Apply strict server-side input validation.
- Avoid dynamically constructed SQL queries where possible.
- Apply least-privilege permissions to database accounts.
- Implement appropriate error handling without exposing database details.
- Conduct security testing for SQL Injection during the SDLC.
- Use secure coding standards.
- Add monitoring and logging for suspicious database query activity.

15. Security Best Practices

- Never trust user input.
- Validate input server-side.

- Use prepared statements.
- Follow least privilege.
- Keep frameworks and database software updated.
- Perform regular vulnerability assessments and penetration testing.
- Monitor abnormal application/database behavior.

16. Conclusion

This practical exercise demonstrated SQL Injection using the intentionally vulnerable DVWA application in an authorized local lab environment. The basic payload ' OR '1'='1 successfully caused the application to return multiple user records instead of a single expected record, demonstrating the security impact of improper input handling on query logic. Supporting TryHackMe activities – including completion of the SQL Injection room and the OWASP Top 10 2025 room – provided theoretical and guided-practical learning that reinforced the concepts applied during the DVWA exercise. Password-hash extraction and UNION-based database enumeration were not demonstrated as completed results in the supplied evidence and are not claimed as part of this task's practical outcome.

17. Skills Gained

- SQL Injection fundamentals
- Web application security testing
- Manual payload testing
- Understanding query manipulation
- UNION-based SQL Injection concepts
- Vulnerability analysis
- Security impact assessment
- Evidence collection
- Technical documentation
- Remediation planning

18. Final Assessment

Task	Week 5 – Task 13 (EH)
Topic	SQL Injection
Lab	DVWA
Practical Result	Basic SQL Injection successfully demonstrated
Evidence	Screenshots provided
TryHackMe SQL Injection Room	100% completion demonstrated by screenshot
OWASP Top 10 2025 Room	100% completion demonstrated by screenshot
Password Hash Extraction	Not demonstrated in supplied evidence
UNION-Based Data Extraction	Not demonstrated as a completed DVWA result in supplied evidence